

```

#kavish_23070521244
# 1. Import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout

# 2. Load dataset
# Example: replace with your actual dataset file
data = pd.read_csv("/content/Housing.csv")

# Separate features (X) and target (y)
X = data.drop("price", axis=1)
y = data["price"]

# Identify categorical columns for one-hot encoding
categorical_cols = X.select_dtypes(include=['object']).columns
X = pd.get_dummies(X, columns=categorical_cols, drop_first=True)

X = X.values # Convert to numpy array after one-hot encoding
y = y.values

# Split into train/test sets
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Standardize features and target
scaler_X = StandardScaler()
scaler_y = StandardScaler()

X_train_scaled = scaler_X.fit_transform(X_train)
X_test_scaled = scaler_X.transform(X_test)

y_train_scaled = scaler_y.fit_transform(y_train.reshape(-1, 1))
y_test_scaled = scaler_y.transform(y_test.reshape(-1, 1))

input_dim = X_train_scaled.shape[1]

# 3. Define architectures

# Model A: Baseline
def build_model_A():
    model = Sequential([
        Dense(128, activation="relu", input_dim=input_dim),
        Dense(64, activation="relu"),
        Dense(32, activation="relu"),
        Dense(1)
    ])
    model.compile(optimizer="adam", loss="mse", metrics=["mae"])
    return model

# Model B: L1/L2 Regularization
from tensorflow.keras.regularizers import l1_l2

def build_model_B():
    model = Sequential([
        Dense(128, activation="relu", input_dim=input_dim, kernel_regularizer=l1_l2(l1=1e-3, l2=1e-3)),
        Dense(64, activation="relu", kernel_regularizer=l1_l2(l1=1e-3, l2=1e-3)),
        Dense(32, activation="relu", kernel_regularizer=l1_l2(l1=1e-3, l2=1e-3)),
        Dense(1)
    ])
    model.compile(optimizer="adam", loss="mse", metrics=["mae"])
    return model

# Model C: Dropout Regularization
def build_model_C():
    model = Sequential([
        Dense(128, activation="relu", input_dim=input_dim),
        Dropout(0.3),
        Dense(64, activation="relu"),
        Dropout(0.3),
        Dense(32, activation="relu"),
        Dense(1)
    ])
    model.compile(optimizer="adam", loss="mse", metrics=["mae"])
    return model

# 4. Train and evaluate models
model_A = build_model_A()

```

```

models = {
    "Baseline": build_model_A(),
    "L1/L2 Regularization": build_model_B(),
    "Dropout Regularization": build_model_C()
}

results = {}

for name, model in models.items():
    print(f"Training {name}...")
    history = model.fit(X_train_scaled, y_train_scaled,
                        validation_data=(X_test_scaled, y_test_scaled),
                        epochs=50, batch_size=32, verbose=0)

    # Predictions
    y_pred_scaled = model.predict(X_test_scaled)
    y_pred = scaler_y.inverse_transform(y_pred_scaled)

    # Metrics
    from sklearn.metrics import mean_absolute_error, mean_squared_error
    mae = mean_absolute_error(y_test, y_pred)
    mse = mean_squared_error(y_test, y_pred)

    results[name] = {"MAE": mae, "MSE": mse, "History": history}

# 5. Comparison table
df_results = pd.DataFrame(results).T[["MAE", "MSE"]]
print(df_results)

# 6. Visualize loss curves
fig, axes = plt.subplots(1, 3, figsize=(18, 5))
for ax, (name, res) in zip(axes, results.items()):
    ax.plot(res["History"].history["loss"], label="Train Loss")
    ax.plot(res["History"].history["val_loss"], label="Validation Loss")
    ax.set_title(f"{name} Loss Curve")
    ax.set_xlabel("Epochs")
    ax.set_ylabel("Loss")
    ax.legend()
plt.show()

```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input\_shape` to `input`
 super().__init__(activity_regularizer=activity_regularizer, **kwargs)

Training Baseline...

4/4 ————— 0s 20ms/step

Training L1/L2 Regularization...

4/4 ————— 0s 34ms/step

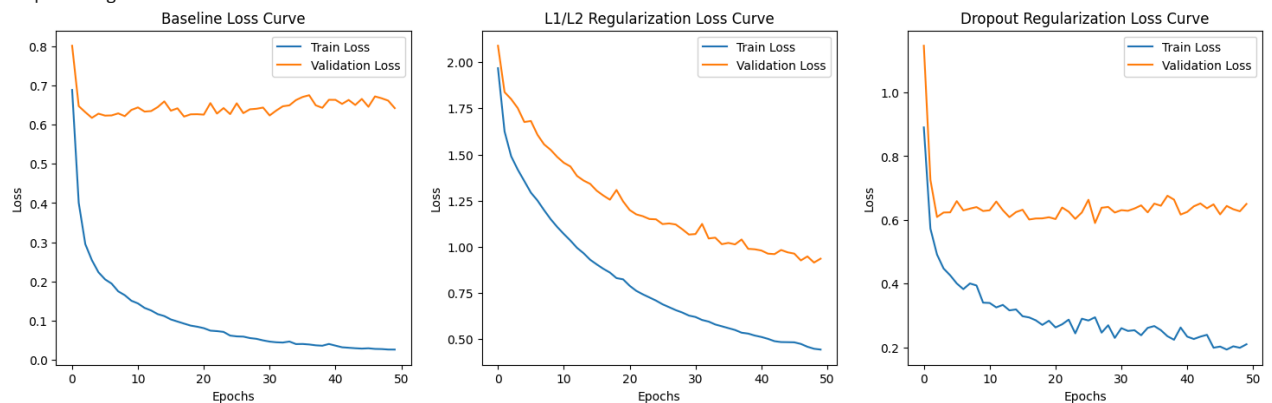
Training Dropout Regularization...

WARNING:tensorflow:5 out of the last 9 calls to <function TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data_

1/4 ————— 0s 58ms/stepWARNING:tensorflow:6 out of the last 12 calls to <function TensorFlowTrainer.make_predi

4/4 ————— 0s 22ms/step

	MAE	MSE
Baseline	1040682.5625	1980911190016.0
L1/L2 Regularization	1002669.0	1844372045824.0
Dropout Regularization	1045991.875	2004679393280.0



Start coding or [generate](#) with AI.

